LEZIONE 10 · BLOCCO 2

VERIFICATION

ED EVALUATION-DRIVEN DEVELOPMENT

Dispensa completa · Corso AI Software Engineering

Evidenze indipendenti, eval harness, grader e release gates

Domain → Requirements → Spec → Test → Agent → Verify

12 ottobre 2026 | Materiale di studio autosufficiente

Guida alla dispensa

Questa dispensa conclude il Blocco 2 del percorso intensivo di AI Software Engineering. Non richiede la rilettura delle slide: include definizioni, spiegazioni, esempi e un laboratorio per progettare la prima valutazione di un coding agent. È progettata per uno studio di circa 40–45 minuti, più il laboratorio pratico.

> **La domanda centrale**
>
> Come dimostriamo che l'output di un agente soddisfi la specification, rispetti i vincoli del sistema e rimanga affidabile quando cambiano prompt, skill, modello o harness?

Obiettivi di apprendimento

- Separare verification, validation e test oracle.
- Progettare evidenze più indipendenti dall'implementatore.
- Distinguere test di codice ed eval su un sistema AI.
- Definire task, trial, grader, trace, outcome e harness.
- Costruire una evaluation suite e scegliere metriche coerenti col rischio.
- Introdurre evaluation gates senza confondere punteggi e correttezza.

Percorso di lettura

- 1. L'oracolo e le evidenze di correttezza.
- 2. Il caso guida: boundary e acceptance test.
- 3. Evaluation-Driven Development e componenti.
- 4. Grader, misure, valutazione del processo e security.
- 5. Laboratorio, template e handoff.

Prerequisiti: Domain Discovery, Requirements Engineering, Spec-Driven Development, TDD/BDD/ATDD, Clean/Hexagonal Architecture, Contract Testing, Property-Based Testing, Context Engineering e harness agentici.

1 · Da test verdi a comportamento corretto

Una dichiarazione dell'agente ("12 test passed") va trattata come un resoconto da verificare, non come una garanzia. Senza sapere quali test sono stati eseguiti, con quali input, rispetto a quali requisiti e in quale ambiente, il messaggio ha un significato limitato.

La correctness va collegata a obbligazioni esplicite: regole del dominio, requisiti, contratti, proprietà, criteri di accettazione e vincoli non funzionali. L'approccio non promette assenza di ogni difetto, ma rende più difficile accettare modifiche senza evidenze pertinenti.

Verification e Validation

Verification	Validation
Il sistema soddisfa gli obblighi specificati?	Il sistema risolve il bisogno reale?
Esempio: non consentire la doppia prenotazione della stessa risorsa	Esempio: permettere risorse diverse nello stesso orario

La verifica è spesso automatizzabile. La validazione richiede un collegamento continuo con gli stakeholder e con il problema. Entrambe avvengono durante l'intero sviluppo. "Formal verification" indica un insieme più specifico di tecniche matematiche e non deve essere confusa con l'uso operativo del termine "verification" in questa lezione.

2 · Test Oracle: la fonte della correttezza

Un test oracle è il meccanismo che decide se il risultato osservato soddisfa un'aspettativa. In un test aritmetico l'oracolo può essere un valore noto; per una regola aziendale può derivare da un esempio approvato; per una API può essere un contratto stabilito; per un sistema complesso può richiedere il controllo dello stato finale.

```
expect(add(2, 3)).toBe(5);

// Il valore 5 è parte dell'oracolo.
```

L'AI può generare sia il codice sia i test. Il rischio compare quando entrambi derivano dalla medesima interpretazione errata della specification. Si produce così una correlazione degli errori: implementazione e prove concordano fra loro, ma sono entrambe scorrette rispetto al bisogno.

> **Principio di authority preservation**
>
> L'agente può proporre test, ma la legittimazione degli expected outcomes deve provenire dal dominio, dai requisiti approvati o da verifiche esterne attendibili.

Come rafforzare l'indipendenza

- Scrivere acceptance examples prima di generare l'implementazione.
- Proteggere i test di verifica dall'agente che modifica il codice.
- Usare riferimenti esterni autorevoli e verificare lo stato del sistema, non la sola risposta testuale.
- Preferire un ambiente pulito per applicare la patch finale e ripetere la verifica.
- Considerare un secondo reviewer come controllo complementare: non equivale a un oracle infallibile.

3 · Una matrice di evidenze, non un solo numero

Una modifica complessa necessita di prove differenti perché i rischi sono differenti. Un test unitario non sostituisce un contract test; il type checking non dimostra la correttezza del dominio; un end-to-end test può non trovare un vincolo di sicurezza violato. Il rischio guida la scelta delle verifiche.

Evidenza	Rischio reso visibile
Unit / acceptance tests	Errore di comportamento e di business
Property-based tests	Controesempi su grandi spazi di input
Contract tests	Incompatibilità consumer/provider
Static analysis / type check	Incoerenza strutturale o vulnerabilità rilevabili staticamente
Integration / E2E	Workflow e collaborazione dei componenti
Review + security checks	Manutenibilità, privilegio, accessi, vincoli operativi

In una pull request critica una buona evidence matrix collega ciascun rischio a una prova che può fallire realmente su quel rischio. Il semplice conteggio dei test non può sostituire questa tracciabilità.

4 · Caso guida: spese di spedizione

Consideriamo una regola di business ipotetica: se il subtotale è inferiore a 50 EUR la spedizione costa 4,99 EUR; da 50 EUR in su è gratuita; il subtotale deve essere un intero in centesimi non negativo. Queste regole sono esplicite e condivise prima dell'implementazione.

```
BR-01: subtotal <  5000  => fee = 499
BR-02: subtotal >= 5000  => fee = 0
BR-03: subtotal <  0     => invalid input
```

Un difetto sul boundary

```
export function shippingFeeCents(subtotal: number): number {
  if (!Number.isInteger(subtotal) || subtotal < 0) {
    throw new RangeError("Invalid subtotal");
  }
  return subtotal > 5000 ? 0 : 499; // BUG: > invece di >=
}
```

Se l'agente testa soltanto subtotal=6000, il risultato è verde. Serve una prova proprio sul valore 5000, derivata da BR-02 e non dall'implementazione.

```
expect(shippingFeeCents(4999)).toBe(499);
expect(shippingFeeCents(5000)).toBe(0);     // rileva il bug
expect(shippingFeeCents(5001)).toBe(0);
expect(() => shippingFeeCents(-1)).toThrow(RangeError);
```

Example tests e property tests

Una proprietà valida è "per ogni subtotale lecito, la fee appartiene all'insieme {0,499}". Ma non individua da sola il bug al boundary. Gli esempi verificano regole puntuali; le proprietà ampliano l'esplorazione; i contratti proteggono l'interfaccia. Vanno usati insieme dove il rischio lo giustifica.

5 · Evaluation-Driven Development

L'Evaluation-Driven Development è un approccio che mette le valutazioni al centro del ciclo di miglioramento di un'applicazione o agente AI. Non ci limitiamo a verificare una singola patch: vogliamo sapere come si comporta una configurazione agentica su una raccolta rappresentativa di task, con più esecuzioni quando necessario.

```
Define criteria -> Build eval suite -> Run -> Grade
          ^                                |
          +--------- Improve ------------+
```

Il ciclo serve per confrontare versioni di prompt, modelli, skill, context file, tool, harness e procedure, mantenendo osservabili regressioni e costi. L'obiettivo non è ottenere un punteggio elevato a qualunque costo, ma avvicinare le metriche ai bisogni reali degli utenti.

Differenza da TDD

TDD	EDD
Guidare l'implementazione di una modifica	Misurare e migliorare il sistema AI
Test specifici, spesso binari	Task rappresentativi, trial e grader multipli
RED -> GREEN -> REFACTOR	EVAL -> ANALYZE -> IMPROVE

Una eval non è necessariamente soggettiva: per coding agent, spesso il miglior grader consiste in test deterministici indipendenti. Allo stesso tempo, un compito creativo può richiedere rubriche e valutazioni umane.

6 · Il vocabolario dell’Evaluation

Termine	Definizione operativa
Task	Attività con input e criteri di successo.
Dataset / suite	Insieme di task rappresentativi da valutare.
Trial	Una singola esecuzione di un task.
Grader	Regola o componente che attribuisce un esito.
Trace / trajectory	Azioni, tool calls, output intermedi e log del trial.
Outcome	Stato reale del sistema dopo il trial.
Agent harness	Struttura che consente al modello di agire.
Eval harness	Infrastruttura che esegue, isola e valuta i trial.

Il task non coincide con l’istruzione testuale: comprende anche il baseline del progetto, le risorse consentite e i criteri di verifica. Il grader non coincide con un LLM: può essere un semplice script. L’outcome non è il messaggio conclusivo dell’agente, ma ciò che è effettivamente accaduto.

7 · Agent Harness ed Eval Harness

L’agent harness può includere tool execution, accesso al repository, skill, subagent, contesto, memoria, worktree e loop operativi. L’eval harness prepara i task, orchestra le esecuzioni, applica i grader, conserva le trace e produce il report. Separare queste responsabilità riduce il rischio di auto-verifica.

```
EVAL HARNESS
  |-- restore clean baseline
  |-- initialize sandbox and task
  |-- launch AGENT HARNESS
  |      |-- reasoning + skills + tools
  |      +-- produce patch and artifacts
  |-- apply patch in trusted environment
  |-- run protected graders
  +-- aggregate report and traces
```

Indipendenza non equivale soltanto a usare due modelli

Un secondo agente può evidenziare problemi che il primo non ha visto, ma può condividere le stesse assunzioni. Un elemento importante è che le aspettative dei grader derivino da fonti autorizzate e che i controlli essenziali possano osservare il comportamento reale. Il worktree Git isola lo stato di lavoro, ma non fornisce da solo isolamento di sicurezza.

8 · Code, Model e Human Graders

Code-based graders

Sono adatti a compilazione, test unitari, acceptance, contratti, controllo dei file modificati, stato del database, type checking e analisi statica. Sono generalmente rapidi e ripetibili, ma valutano ciò che è stato codificato in regole osservabili.

```
import subprocess

def verify(repo_path):
    proc = subprocess.run(
        ["npm", "run", "test:verify"],
        cwd=repo_path, capture_output=True, text=True,
        timeout=120, check=False,
    )
    return {"passed": proc.returncode == 0,
            "stdout": proc.stdout, "stderr": proc.stderr}
```

Questo esempio dimostra il meccanismo di grading. Un harness reale non deve eseguire codice non attendibile con credenziali di produzione o senza isolamento di processo e filesystem.

Model-based graders

Un LLM-as-a-judge è adatto a criteri che richiedono lettura di un diff, discussione di qualità architetturale, riconoscimento di eccessi di complessità o valutazione del rispetto di una rubrica. È meno adatto come unica prova della correttezza tecnica quando esistono verifiche deterministiche.

Una rubrica utile impone di citare righe e comportamento verificato, distingue sicurezza da stile, e prevede la risposta “evidenza insufficiente”. Per una calibrazione seria si confrontano periodicamente i giudizi del modello con quelli di revisori esperti.

Human graders

Gli esperti di dominio verificano aspetti non formalizzati e validano la soluzione rispetto al problema. Architect e security reviewer valutano rispettivamente coerenza dei boundary e rischi operativi. La review umana è preziosa, ma non sostituisce prove automatizzate ripetibili.

9 · Outcome vs Trajectory

Un agente può dichiarare “refund completed”, ma il risultato rilevante è l’esistenza di un rimborso correttamente registrato. La trace mostra come l’agente è arrivato al risultato: file letti, chiamate API, modifiche, errori, retry. L’outcome descrive invece lo stato finale verificabile.

Due vincoli complementari

Successo funzionale: l’azione richiesta è stata eseguita. Conformità operativa: l’agente ha rispettato limiti, permessi e strumenti autorizzati. Un esito corretto non cancella una violazione di sicurezza.

Per un coding agent possiamo controllare non solo test e patch, ma anche l’assenza di modifiche ai test fidati, di disattivazioni dei controlli e di accessi a dati o servizi esterni non necessari. Non tutte le tool call devono essere prescritte: imporre una traiettoria unica può penalizzare soluzioni alternative legittime.

10 · Capability Evals e Regression Evals

Capability eval	Regression eval
Misura i limiti e le capacità in crescita	Protegge comportamenti già dimostrati
Può contenere task attualmente difficili	Preferisce task da superare con elevata affidabilità
Guidano la roadmap del miglioramento	Rilevano peggioramenti tra release

Un task precedentemente difficile, una volta risolto in modo ripetibile, può entrare nella regression suite. Un fallimento scoperto in produzione dovrebbe diventare, se rappresentativo e verificabile, un nuovo caso di regressione. In questo modo il sistema accumula evidenze nel tempo.

11 · Variabilità e ripetibilità dei trial

Una esecuzione riuscita dimostra soltanto un successo osservato. Non garantisce che il task sia sempre risolto: l’output può variare per campionamento, ordine degli strumenti, ambiente, contesto o dipendenze. Occorre registrare numero di task, trial, versioni e condizioni. L’8/10 osservato su un piccolo campione non si trasferisce automaticamente a ogni progetto software.

Due metriche da non confondere

pass@k: almeno un successo tra k tentativi. È utile quando è possibile generare diverse proposte e selezionare una soluzione valida.

pass^k: tutti i k tentativi hanno successo. È utile quando l’affidabilità ripetuta è importante.

Le due metriche non sono intercambiabili; un sistema che trova occasionalmente la soluzione non è necessariamente un sistema consistente. Per confronti seri è utile considerare variabilità, dimensione campionaria, baseline stabile e incertezza.

12 · Progettare il primo dataset

La progettazione inizia dai rischi reali. Per un coding agent possono essere utili task di boundary logic, autorizzazioni, backward compatibility, refactoring, input parsing e idempotenza. Ogni task dovrebbe contenere un problema raggiungibile, un baseline di repository noto e una soluzione di riferimento o un'altra prova che i grader siano corretti.

Task	Grader prioritario
Boundary bug	Acceptance indipendente
Autorizzazione	Accesso negato + security
API backward compatibility	Contract tests
Refactoring	Differential/regression tests
Parser	Property-based tests
Retry di pagamento	Idempotenza + end-state checks

```
task:
  id: shipping-boundary-001
  baseline: BASELINE_COMMIT
  instructions: task.md
  trusted_tests: trusted/acceptance
  limits:
    network_access: false
    timeout_seconds: 600
```

Questo YAML è una convenzione didattica. Non è un formato universale o la configurazione ufficiale di un prodotto. Per ogni test è utile conservare un expected outcome spiegabile da un domain expert, altrimenti il grader rischia di testare un'assunzione arbitraria.

13 · Metriche e Evaluation Gates

Le misure più utili per un coding agent includono correttezza, regressioni, violazioni dei vincoli, compatibilità dei contratti, durata, costo e qualità architetturale. Tuttavia non tutti gli indicatori devono contribuire a una media unica: alcuni sono vincoli necessari.

```
HARD GATES:  auth tests, critical contracts, security
ADVISORY:    maintainability notes, cost, style
```

Un evaluation gate può impedire il merge quando un controllo obbligatorio fallisce. I required status checks e i code scanning checks di GitHub sono esempi di meccanismi infrastrutturali su cui innestare regole di verifica. Un grader qualitativo non calibrato, invece, è spesso più utile come

segnale da esaminare che come vincolo assoluto.

14 · Failure Modes delle evaluation

Metric gaming

Se si misura soltanto il numero di test verdi, eliminare un test difficile migliora il punteggio pur peggiorando il prodotto. L'indicatore deve essere collegato a un obiettivo di qualità e non sostituirlo.

Contamination e overfitting

Quando i casi e le risposte del benchmark sono già noti al sistema, la valutazione può misurare memorizzazione o ottimizzazione sul dataset. Separare task di sviluppo e task riservati alla verifica può migliorare l'affidabilità del confronto.

Flakiness dell'infrastruttura

Reti, clock, dipendenze instabili, race condition del test e servizi esterni non disponibili introducono rumore. L'infrastruttura deve distinguere un errore del trial da un errore dell'ambiente.

Judge bias

LLM reviewer e coding agent possono condividere omissioni; la valutazione automatica va calibrata e non dovrebbe essere l'unica evidenza su regole deterministiche o violazioni gravi.

Task impossibili o specifica ambigua

Un task può risultare irrisolvibile perché la documentazione non descrive ciò che il grader pretende. Una reference solution o una revisione da parte di esperti aiuta a validare il dataset stesso.

Processo sproporzionato

Una modifica cosmetica non necessita dello stesso investimento valutativo di un cambio di autorizzazioni o gestione pagamenti. La verification strategy deve seguire rischio e blast radius.

15 · Security della verifica agentica

Un agent harness può dare accesso a shell, filesystem, rete, Git, cloud e database. Se l'agente controlla i propri test o possiede credenziali non necessarie, un successo funzionale può mascherare un danno. Sono importanti least privilege, workspace isolato, token a vita breve, approvazioni per azioni sensibili, registrazione delle tool call e separazione dell'ambiente di grading.

OWASP segnala per gli agenti rischi di hijacking del comportamento, tool misuse e privilege abuse. Le contromisure devono essere tecniche e operative: non basta chiedere all'agente di comportarsi bene.

16 · Collegamento con Matt Pocock

Il workflow grill-with-docs / grill-me → domain-modeling → glossario / ADR → to-spec → to-tickets → implement / TDD → code-review agisce sul singolo cambiamento: migliora comprensione, pianificazione, esecuzione e review. L'EDD agisce a un livello superiore, misurando se il processo di produzione del codice migliori al cambiare di skill, prompt, tools e harness.

Nel repository pubblico aggiornato di Matt Pocock la convenzione per il glossario condiviso utilizza GLOSSARY.md; versioni precedenti impiegavano CONTEXT.md. Questo non cambia il principio: un artefatto contiene conoscenza del dominio e rende esplicite le assunzioni.

Diagnosi prima del cambio di modello

Un insuccesso può dipendere dal modello, ma anche da task ambigui, mancata navigazione del repository, una skill inefficace, un context file impreciso, un test difettoso o un boundary poco chiaro.

17 · Laboratorio guidato: eval harness minimale

Parte A — Definire dominio, spec e trusted tests

- Usa la funzione shippingFeeCents con importi in centesimi e le quattro regole dell'esercizio.
- Scrivi acceptance tests indipendenti sui valori 4999, 5000, 5001 e -1.
- Conserva i trusted tests fuori dall'area modificabile dall'agente.

Parte B — Trial riproducibili

- Ripristina il baseline del repository prima di ogni tentativo.
- Fornisci all'agente la stessa specification e limiti di tool.
- Acquisisci commit iniziale, patch, tool calls significative e durata.
- Applica la patch a un ambiente pulito, poi esegui i trusted tests.

Parte C — Grading e miglioramento

- Ripeti tre volte lo stesso task; registra successo/fallimento e causa.
- Modifica UNA sola variabile: aggiungi l'istruzione di identificare i boundary prima di programmare.
- Confronta i trial successivi e osserva se l'errore al valore 5000 ricompare.
- Trasforma un nuovo failure mode in un test regressivo.

```
{
  "task_id": "shipping-001",
  "baseline": "example-commit",
  "acceptance_passed": false,
  "regression_passed": true,
  "contract_passed": null,
  "verdict": "FAILED"
}
```

Il valore null indica controllo non eseguito o non applicabile, non successo. Il report finale dovrebbe includere evidenze e, dove necessario, eccezioni autorizzate e questioni aperte.

18 · Check finale di comprensione

- Sai distinguere output dichiarato, outcome verificato e trajectory?
- Sai spiegare perché test generati dalla stessa interpretazione possono produrre falso successo?
- Sai scegliere un grader deterministico quando esiste un criterio eseguibile?
- Sai distinguere capability suite e regression suite?
- Sai progettare una eval senza confondere 8/10 osservato con una garanzia generale?
- Sai dire quali controlli diventano hard gates e perché?

Takeaway

L’agente implementa; l’ingegneria conserva l’autorità sui criteri e verifica che il risultato li soddisfi. EDD aggiunge la capacità di misurare e migliorare nel tempo l’intero processo agentico.

19 · Fonti e letture autorevoli

Anthropic, “Demystifying evals for AI agents” (9 gennaio 2026)

https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

Google, “Evaluation-driven development” (29 gennaio 2026)

https://web.dev/learn/ai/evaluation-driven-development

OpenAI, “Evaluation best practices” (consultato ottobre 2026)

https://developers.openai.com/api/docs/guides/evaluation-best-practices

Martin Fowler / Ham Vocke, “The Practical Test Pyramid”

https://martinfowler.com/articles/practical-test-pyramid.html

Matt Pocock, “Skills for Real Engineers”

https://github.com/mattpocock/skills

OWASP Gen AI Security Project, Agentic Security Top 10 (dicembre 2025)

https://genai.owasp.org/2025/12/09/owasp-genai-security-project-releases-top-10-risks-and-mitigations-for-agentic-ai-security/

GitHub Documentation, Required status checks / Merge protection

https://docs.github.com/en/pull-requests/how-tos/merge-and-close-pull-requests/troubleshooting-required-status-checks

Nota di aggiornamento: alla data del 12 ottobre 2026, OpenAI segnala la dismissione della piattaforma Evals storica (read-only dal 31 ottobre 2026 e chiusura prevista al 30 novembre 2026). Il metodo EDD resta indipendente da tale prodotto.

Conclusione del Blocco 2

Nel Blocco 2 sono state studiate: Lezione 6 — Clean/Hexagonal Architecture; Lezione 7 — Contract Testing e Property-Based Testing; Lezione 8 — Context Engineering; Lezione 9 — Skills, subagent, worktree, Superpowers e Ralph loops; Lezione 10 — Verification ed Evaluation-Driven Development.

Il passaggio al Blocco 3 avverrà con un handoff completo in una nuova chat. Il corso potrà approfondire DDD avanzato, architettura dei sistemi multi-agente, sicurezza e una pipeline operativa end-to-end con verification harness.